Open in Colab

1. Model parameters vs hyperparameters

a. Model Parameters

- **Definition**: Parameters are the values that the model **learns from the training data** during the training process.
- **Key Characteristics**:
  - They are **internal to the model** and adjusted automatically during training.
  - They directly influence the predictions or output of the model.
  - Their values depend on the training data and optimization process.
- **Examples**:
  - In a **Linear Regression** model:
    - The coefficients (w) and the intercept (b) are parameters.
  - In a **Neural Network**:
    - The weights and biases in each layer are parameters.
  - In **K-Means** clustering:
    - The **centroids** of the clusters are parameters.

b. Model Hyperparameters

- **Definition**: Hyperparameters are **external settings** for the model that are **not learned from the data** but are set **before training** begins.
- **Key Characteristics**:
  - They are used to control the learning process or the structure of the model.
  - They are tuned **manually** (or via search methods like grid search) to optimize the model's performance.
  - Hyperparameters must be chosen carefully because they can significantly impact the model's ability to generalize.
- **Examples**:
  - In **K-Means** clustering:
    - The number of clusters (k) is a hyperparameter.
  - In **DBSCAN**:
    - Epsilon (ϵ) and `min_samples` are hyperparameters.
  - In **Neural Networks**:
    - The number of layers, number of neurons, learning rate, and batch size are hyperparameters.
  - In **Decision Trees**:
    - Maximum tree depth (`max_depth`), minimum samples per split (`min_samples_split`), etc., are hyperparameters.

Summary: Key Differences

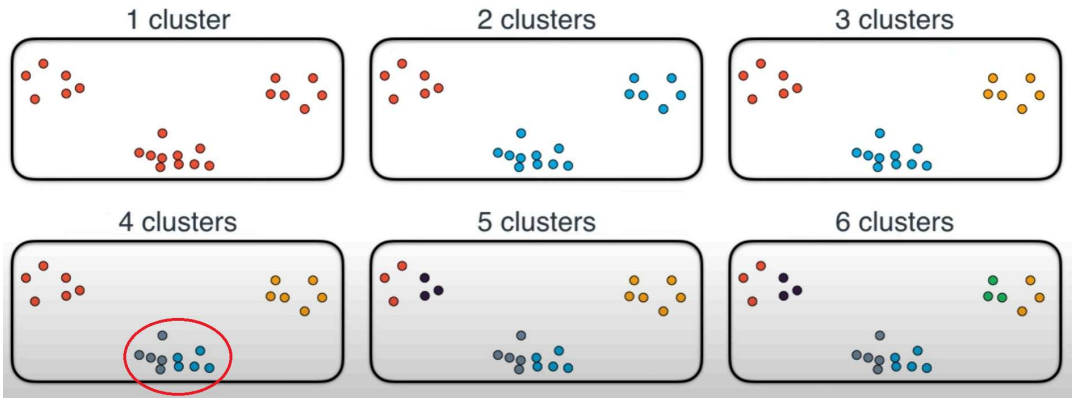
Feature	Model Parameters	Hyperparameters
Definition	Learned from the data during training	Set manually before training
Adjustable?	Adjusted by the optimization algorithm	Set by the user or via tuning techniques
Examples	Weights in Neural Networks	Learning rate, number of layers
Model Training	Updated during training	Remain fixed during training

---

2. Hyperparameters (Tunning/Selection) For K-Means Clustering Algorithm

Choosing the Right K For K-Means Clustering (The Elbow Method)

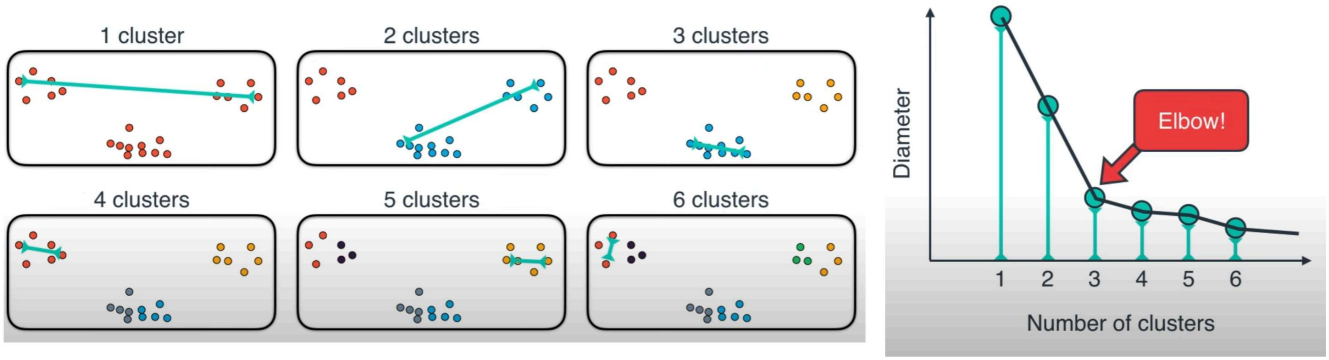
One challenge with K-Means is selecting the appropriate number of clusters K.
We can try different cluster numbers and plot them and eye ball the goodness of the created clusters.
For example, in the image below, we can notice that we started to create unclear clusters **starting from K = 4**.



The **Elbow Method** is a technique used to determine the optimal K by plotting the **within-cluster sum of squares (WCSS)** against different values of K.

Steps of the Elbow Method:

1. Run K-Means for a range of K values.
2. Plot the WCSS for each K.
3. Look for the "elbow" point in the plot, where the decrease in WCSS slows down. This is the ideal K.



YouTube

Elbow Method in Python

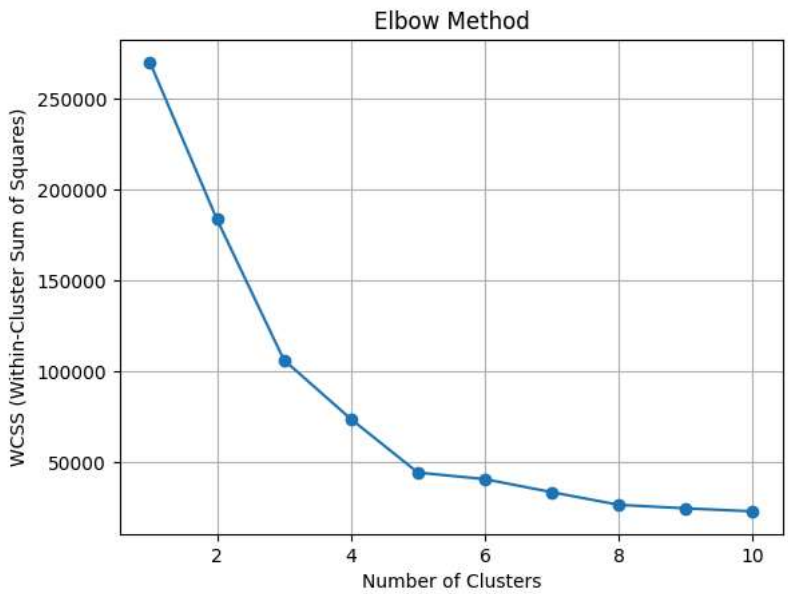
Import Libraries and Data

```
# Import necessary libraries
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Step 1: Get the data
df = pd.read_csv('https://raw.githubusercontent.com/msfasha/307304-Data-Mining/refs/heads/main/20242/datasets/mall_customers.csv')
features = df[["Annual Income (k$)", "Spending Score (1-100)"]]
```

Implement Elbow Method

```
# Step 1: Compute WCSS for the given dataset
wcss = []

# Using the features "Annual Income (k$)" and "Spending Score (1-100)"
for k in range(1, 11):  # Testing cluster counts from 1 to 10
```

```
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(features)  # Fit on the selected features
    wcss.append(kmeans.inertia_)  # Inertia is the WCSS

# Step 2: Plot the elbow graph
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.grid()
plt.show()
```



The elbow chart above illustrates the process of determining the optimal number of clusters for customer segmentation based on the features "Annual Income (k$)" and "Spending Score (1-100)." Key observations include:

- **Purpose:** The chart plots the Within-Cluster Sum of Squares (WCSS) against the number of clusters to identify the point where adding more clusters does not significantly reduce WCSS.
- **Trend:** As the number of clusters increases, WCSS decreases due to better compactness within each cluster.
- **Elbow Point:**
  - The "elbow" is observed at **4 or 5 clusters**, where the rate of decrease in WCSS slows down noticeably.
  - This indicates the optimal number of clusters, balancing simplicity and effectiveness.
- **Interpretation:** Selecting 4 or 5 clusters will likely result in meaningful segmentation of customers into distinct groups for analysis or targeted strategies.

---

3. Hyperparameters (Tunning/Selection) For DBSCAN Algorithm

Determining the **hyperparameters** for the **DBSCAN** algorithm, namely **epsilon** (ϵ) and **minimum points** (`min_samples`), can be challenging. However, there are systematic methods to estimate these values effectively before fine-tuning through trial and error.

a. Determining min_samples

The **min_samples** parameter defines the minimum number of points (including the point itself) that must exist in a neighborhood of radius ϵ for the point to be considered a **core point**.

Techniques to Estimate min_samples

i. **Rule of Thumb**:

- Start with:

$$\text{min_samples} = \text{number of features} + 1$$

- For example:

- 2D data → `min_samples = 3`.
- 3D data → `min_samples = 4`.

ii. **Domain Knowledge**:

- If you know the minimum density required for clusters, you can set `min_samples` accordingly.
- Example: In customer segmentation, you might set `min_samples` to 5 or 10 to define a minimum cluster size.

iii. **Experimentation**:

- Run DBSCAN with a range of `min_samples` values (e.g., 3–10) and evaluate the resulting clusters visually or using metrics like the Silhouette Score.

b. Determining Epsilon (ϵ)

The **epsilon** parameter defines the neighborhood radius around each point. Points within this radius are considered neighbors.

Techniques to Estimate Epsilon

i. **k-Distance Plot (k-NN Distance Plot)**:

- Compute the distances to the **k-th nearest neighbor** for all points (where k = min_samples).
- Sort the distances in ascending order and plot them.
- Look for the "**elbow point**" where the slope changes significantly. This is a good candidate for ϵ.
- The elbow point in the plot suggests the optimal ϵ.
- If the plot lacks a clear elbow, test a range of values manually.

ii. **Domain Knowledge**:

- If you know the scale or spread of the data, you can estimate ϵ based on typical distances between points.
- Example: In geographic data, ϵ could correspond to a known radius (e.g., kilometers).

iii. **Distance Metrics**:

- Inspect the pairwise distances between points to understand the average neighborhood radius.

c. Combining ϵ and min_samples

i. **Iterative Refinement**:

- Start with the **k-distance plot** to find an initial estimate for ϵ.
- Use `min_samples = 4` or a value based on the number of features.
- Run DBSCAN and evaluate the results.
- Adjust ϵ and `min_samples` iteratively to fine-tune the clustering.

ii. **Visualization**:

- Visualize the clusters using scatter plots to ensure they align with expectations.
- Points labeled as **-1 (noise)** should make sense intuitively.

d. Validation Metrics

After selecting ϵ and `min_samples`, validate the clustering using metrics:
i. **Number of Clusters**: Check if the number of clusters is reasonable for your data.

ii. **Visual Inspection**: Plot the clusters and noise points to ensure they align with the data's structure.

iii. **Silhouette Score**: Measures how well each point fits into its cluster.

```
from sklearn.metrics import silhouette_score
silhouette_score(X, dbscan_labels)
```

Summary

a. Use the **k-distance plot** to estimate ϵ.
b. Start with `min_samples = 4` (rule of thumb) or based on domain knowledge.
c. Fine-tune ϵ and `min_samples` iteratively, evaluating results visually and using metrics like the Silhouette Score.

Practical Example using Python

In this example, we will use the Mall Customers Dataset which we used in KMEANs clustering section and we will try to determine the initial hyperparameters using the methods discussed above.

Import the Libraries and the Data

We will import mall customers data and select two features (Annual Income and Spending Score) and use them to create our clustersso as to make plotting easier for learning purposes.

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors


# Get the data
df = pd.read_csv('https://raw.githubusercontent.com/msfasha/307304-Data-Mining/refs/heads/main/20242/datasets/mall_customers.csv')
df = df[["Annual Income (k$)", "Spending Score (1-100)"]]

df
```

	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40
...	...	...
195	120	79
196	126	28
197	126	74
198	137	18
199	137	83

200 rows × 2 columns

Compute distances to the k-th nearest neighbor

```
neighbors = NearestNeighbors(n_neighbors=4)  # Use min_samples = 4
neighbors_fit = neighbors.fit(df)
distances, _ = neighbors_fit.kneighbors(df)
```

Show the neighbors distances for the first 5 customer records

```
distances[:5,:]
```

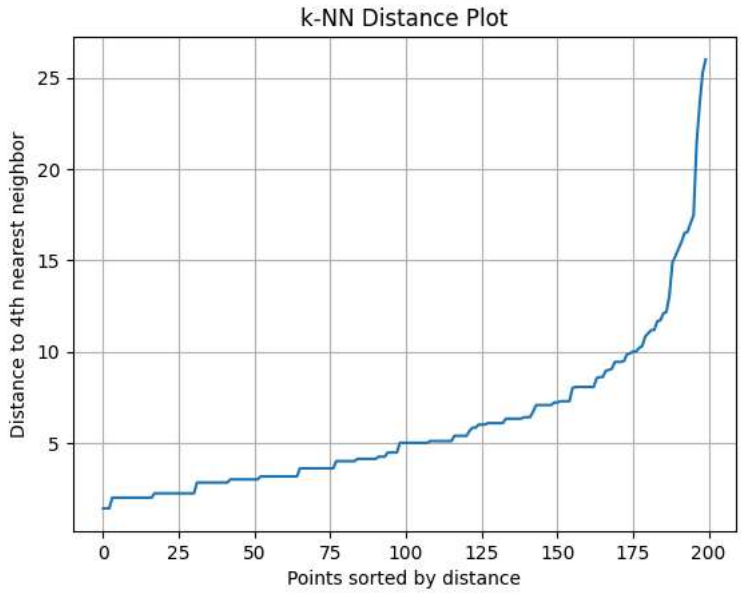
```
array([[0.        , 2.23606798, 7.21110255, 9.8488578 ],
       [0.        , 4.12310563, 5.38516481, 5.38516481],
       [0.        , 2.        , 4.24264069, 8.06225775],
       [0.        , 1.41421356, 4.        , 4.12310563],
       [0.        , 2.23606798, 6.40312424, 8.60232527]])
```

Sort the distances to the k-th nearest neighbor

```
distances = np.sort(distances[:, 3])  # 3rd neighbor (0-based index)
```

Plot The Distances and Look for the Elbow

```
plt.plot(distances)
plt.title("k-NN Distance Plot")
plt.xlabel("Points sorted by distance")
plt.ylabel("Distance to 4th nearest neighbor")
plt.grid()
plt.show()
```



Run DBSCAN using the Distance Value based on the previous Elbow Plot

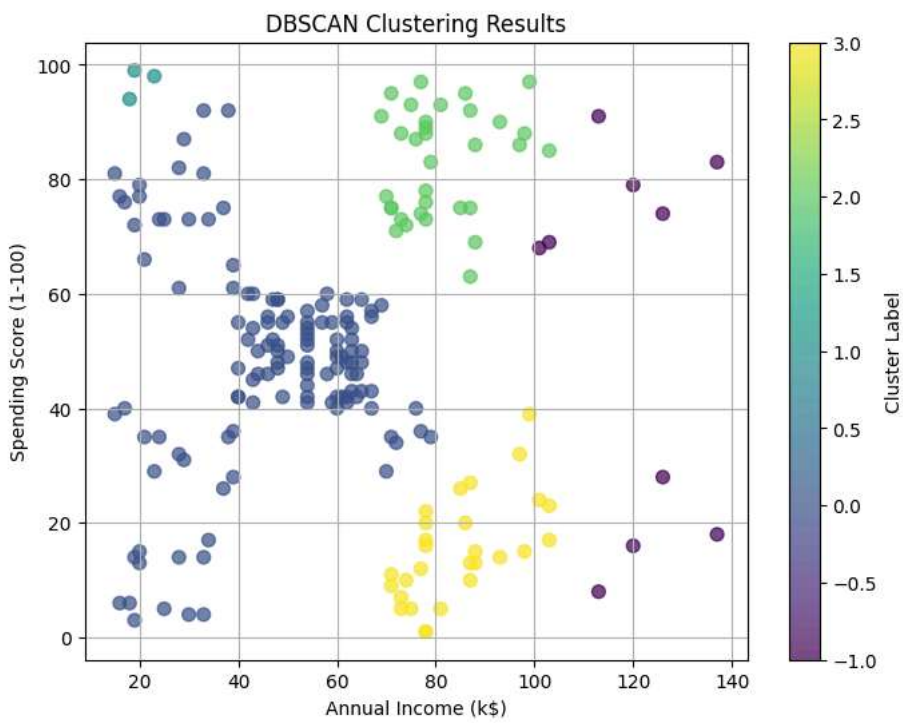
```
# Import necessary libraries
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# Run DBSCAN
epsilon = 10  # Based on the K-NN distance plot
min_samples = 3  # number of features + 1
dbscan = DBSCAN(eps=epsilon, min_samples=min_samples)
labels = dbscan.fit_predict(features)

# Add cluster labels to the original dataframe
df['Cluster'] = labels
df

```

	Annual Income (k$)	Spending Score (1-100)	Cluster
0	15	39	0
1	15	81	0
2	16	6	0
3	16	77	0
4	17	40	0
...	...	...	...
195	120	79	-1
196	126	28	-1
197	126	74	-1
198	137	18	-1
199	137	83	-1

200 rows × 3 columns

Visualize the clustering results

```
plt.figure(figsize=(8, 6))
plt.scatter(df["Annual Income (k$)"], df["Spending Score (1-100)"], c=labels, cmap='viridis', s=50, alpha=0.7)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("DBSCAN Clustering Results")
plt.colorbar(label="Cluster Label")
plt.grid()
plt.show()
```



Print summary

Print out the number of created clusters and the number of noise points.

```
num_clusters = len(set(labels)) - (1 if -1 in labels else 0)  # Exclude noise label (-1)
print(f"Number of clusters (excluding noise): {num_clusters}")
print(f"Number of noise points: {list(labels).count(-1)}")
```

```
Number of clusters (excluding noise): 4
Number of noise points: 10
```

Validate Clustering with Silhouette Score

The **silhouette score** is a metric used to evaluate the quality of clustering. It gives an idea of how similar a point is to its own cluster compared to other clusters.

What is the silhouette measure?

- For each data point, the silhouette score measures two things:
  1. **Cohesion (a):** How close the point is to other points in its own cluster.
  2. **Separation (b):** How far the point is from points in the nearest other cluster.
- The silhouette score for a point is calculated as:

$$s = \frac{b - a}{\max(a, b)}$$

  where:
  - (a): Average distance to other points in the same cluster (within-cluster distance).
  - (b): Average distance to points in the nearest other cluster (nearest-cluster distance).

- The score ranges between:
  - **+1**: The point is very close to its own cluster and far from other clusters (well-clustered).
  - **0**: The point is on or near the boundary between two clusters.
  - **-1**: The point is closer to another cluster than its own (poorly clustered).

```
from sklearn.metrics import silhouette_score

# Compute Silhouette Score
if len(set(labels)) > 1:  # Avoid silhouette score error for single cluster
    score = silhouette_score(features, labels)
    print(f"Silhouette Score: {score:.2f}")
else:
    print("Silhouette Score cannot be calculated for a single cluster.")
```

```
Silhouette Score: 0.36
```

Score = 0.36 — Is it Good?

A score of 0.36 is:

Moderate clustering quality.

It suggests that there is some structure, but the clusters may not be well-separated or clearly defined.

General Benchmarks:

- > 0.7: Strong structure, well-separated clusters.
- 0.5 – 0.7: Reasonable structure.
- 0.25 – 0.5: Weak structure (yours falls here).
- < 0.25: Poor structure.

So, 0.36 is acceptable, especially if your data is complex or noisy, but there is likely room for improvement (e.g., tuning number of clusters, trying different features, or using another clustering algorithm).